Docker Compose:

Administration des conteneurs

Departement : Génie Informatique

Spécialité : Développement Web

Y.NDIONE (Formateur certifié) Mail: youcomputing@gmail.com

Chapitre 3

docker
Compose

Docker Compose : Orchestration multi-conteneurs

O P : A la fin de ce chapitre , l'étudiant sera capable de :

- ✓ Comprendre L'utilité de Docker Compose
- ✓ Déployer une application multi-services
- ✓ Connecter une application Flask à une base de données
- ✓ Utiliser un fichier docker-compose.yml
- ✓ Lancer toute une architecture avec une seule commande

1 Etat des lieux

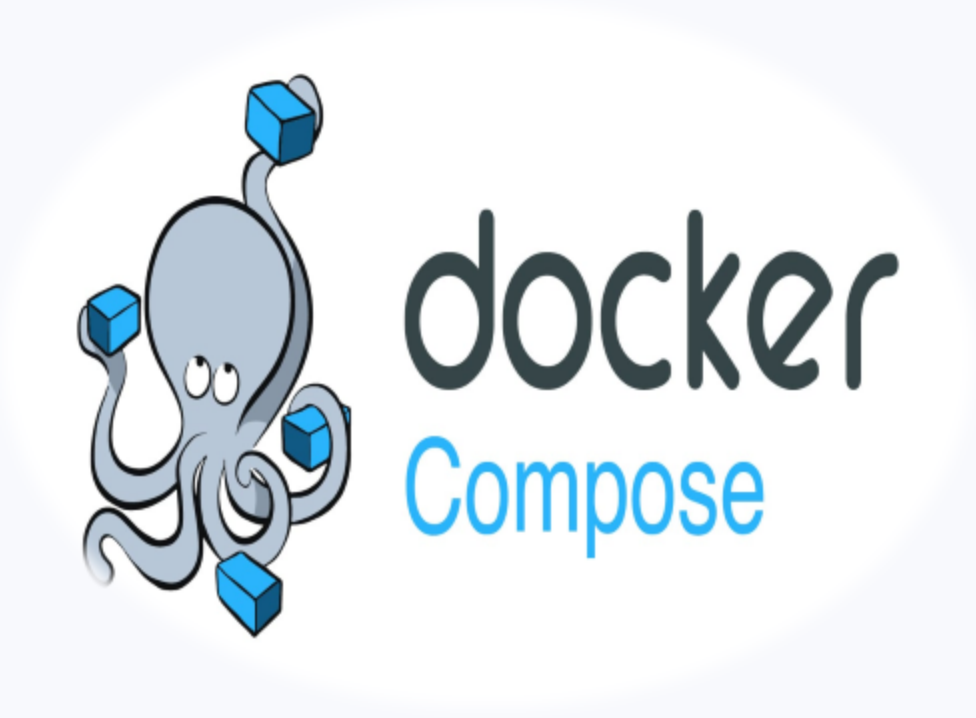
Jusqu'ici : Création et administration d'une application via un seul conteneur (création d'une image personnalisée, création et configuration basique d'un conteneur (mapping des ports , volumes simples)).

Maintenant : Imaginons une application qui repose sur plusieurs services complémentaires, comme PostgreSQL , un cache Redis etc. Si chaque composant est conteneurisé individuellement, plusieurs problèmes émergent:

- Commandes très longues
- Gestion manuelle du réseau
- Mauvaise lisibilité et défaut dans la maintenabilité
- Difficulté dans la gestion des versions de l'application

NB : Basculer vers Docker Compose, une solution pertinente

2 Qu'est ce que Docker Compose ?



Docker compose est un outil open source développé par Docker pour la définition et la gestion d'une application multi-conteneur.

Docker compose repose sur une approche déclarative permettant de spécifier dans un seul fichier YML généralement nommé docker-compose.yml , l'ensemble des services , réseaux et volumes nécessaires au fonctionnement d'une application.

3

Structure d'un fichier docker-compose.yml

```yaml
version: '3.9'

services:
  web:
  db:

volumes:
```

Version du format

Liste des conteneurs

Stockage persistant

Un service dans un docker-compose représente un conteneur ou groupe de conteneurs identiques qui execute la même image Docker pour remplir une fonction spécifique au sein d'une application

Exemple : Exemple d'une application qui utilise 4 services (react – web – db – cache) : docker-compose (Développement)

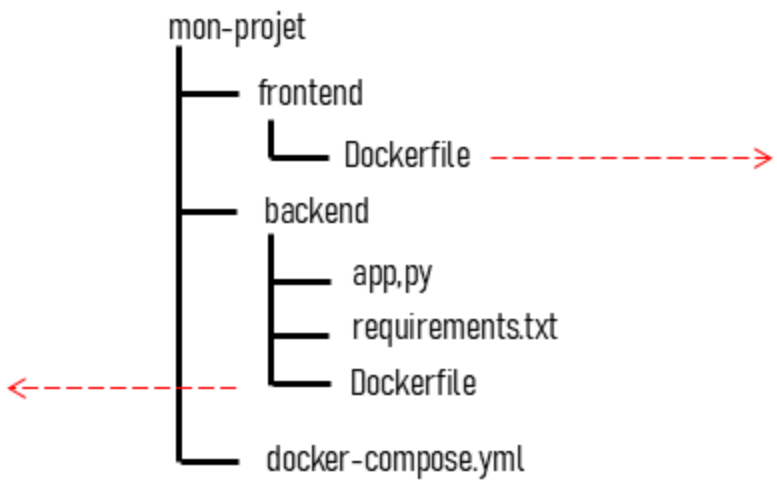
```yaml
version: "3.9"

services:

  frontend:
    build: ./frontend
    container_name: react_frontend_dev
    command: npm start
    ports:
      - "3000:3000"
    volumes:
      - ./frontend:/app
      - /app/node_modules
    environment:
      - CHOKIDAR_USEPOLLING=true
    depends_on:
      - web
    networks:
      - backend

  web:
    build: ./backend
    container_name: flask_backend_dev
    command: python app.py
    ports:
      - "5000:5000"
    volumes:
      - ./backend:/app
    environment:
      - FLASK_ENV=development
      - FLASK_DEBUG=1
      - DATABASE_URL=postgresql://postgres:password@db:5432/mydb
      - REDIS_URL=redis://cache:6379
    depends_on:
      - db
      - cache
    networks:
      - backend
```

```
mon-projet
├── frontend
│   └── Dockerfile
├── backend
│   ├── app.py
│   ├── requirements.txt
│   └── Dockerfile
└── docker-compose.yml
```

```yaml
  db:
    image: postgres:15
    container_name: postgres_db_dev
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: password
      POSTGRES_DB: mydb
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
    networks:
      - backend

  cache:
    image: redis:7
    container_name: redis_cache_dev
    ports:
      - "6379:6379"
    networks:
      - backend

volumes:
  postgres_data:

networks:
  backend:
```

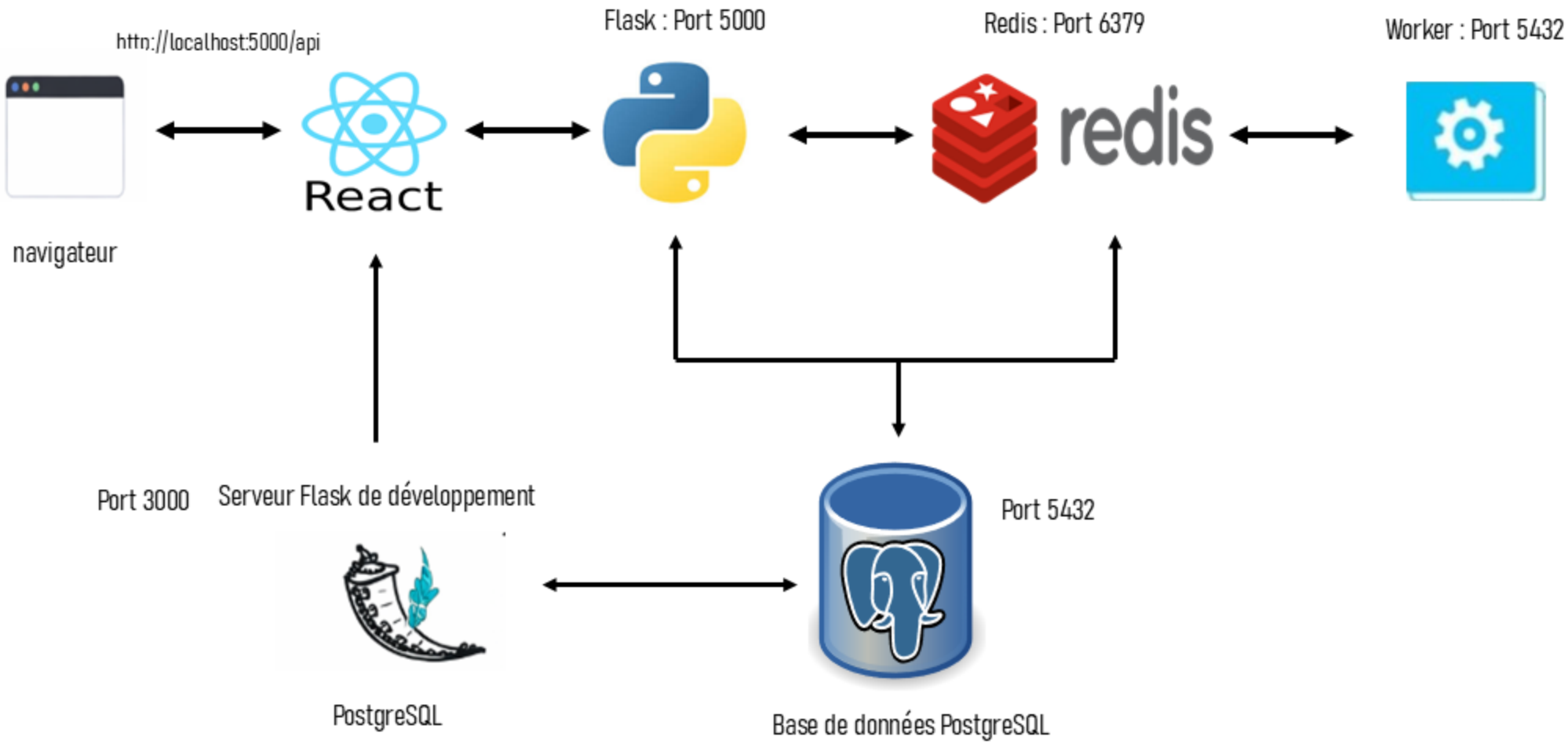
Exemple : Exemple d'une application qui utilise 4 services (react – web – db – cache) : Dockerfile (Développement)

```
FROM python:3.11-alpine

ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 5000

CMD ["python", "app.py"]
```




```
FROM node:18

WORKDIR /app

COPY package.json package-lock.json ./

RUN npm install

COPY . .

EXPOSE 3000

CMD ["npm", "start"]
```

Architecture



Exemple : Exemple d'une application qui utilise 5 services (react – web – db – cache et nginx) : docker-compose (Production)

1

```yaml
version: "3.9"

services:

  frontend:
    build: ./frontend
    container_name: react_frontend
    restart: always
    depends_on:
      - web
    networks:
      - backend

  web:
    build: ./backend
    container_name: flask_backend
    restart: always
    environment:
      - DATABASE_URL=postgresql://postgres:password@db:5432/mydb
      - REDIS_URL=redis://cache:6379
    depends_on:
      - db
      - cache
    networks:
      - backend
```

2

```yaml
db:
  image: postgres:15
  container_name: postgres_db
  restart: always
  environment:
    POSTGRES_USER: postgres
    POSTGRES_PASSWORD: password
    POSTGRES_DB: mydb
  volumes:
    - postgres_data:/var/lib/postgresql/data
  networks:
    - backend
```

```
mon-projet
├── frontend
│   └── Dockerfile
├── backend
│   ├── app.py
│   ├── requirements.txt
│   └── Dockerfile
├── nginx
│   └── default.conf
└── docker-compose.yml
```

3

```yaml
cache:
  image: redis:7
  container_name: redis_cache
  restart: always
  networks:
    - backend

nginx:
  image: nginx:latest
  container_name: reverse_proxy
  ports:
    - "80:80"
  volumes:
    - ./nginx/default.conf:/etc/nginx/conf.d/default.conf
  depends_on:
    - frontend
    - web
  networks:
    - backend

volumes:
  postgres_data:

networks:
  backend:
```

Bon à savoir ! : Exemple d'une application

En développement

Pour lancer le serveur Flask

→ python app.py

En Production

Pour lancer le serveur Flask

→ python app.py

→ gunicorn app:app -w **4** -b 0.0.0.0:5000

-w 4	Workers multiprocess
app:app	fichier app.py + Objet Flask
-b 0.0.0.0:5000	Ecoute sur le réseau
gunicorn	Serveur WSGI pour Python (en production)

NB : gunicorn est utilisé derrière un reverse proxy comme nginx

Avantage

- ✓ Gestion de plusieurs simultanées
- ✓ Gain en performance
- ✓ Meilleure gestion des workers

Exemple : Exemple d'une application qui utilise 5 services (react – web – db – cache et nginx) : Dockerfile (Production)

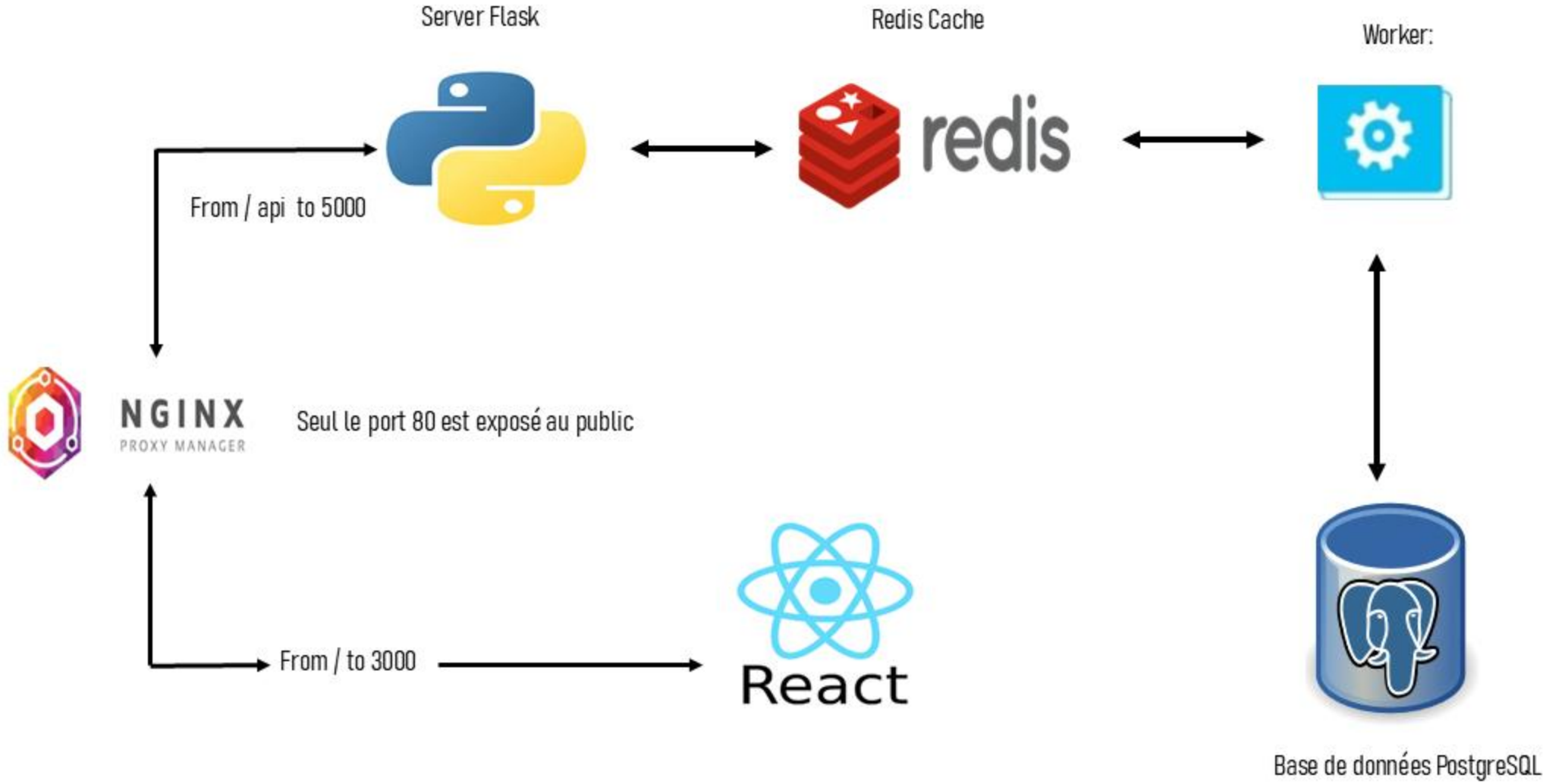
```
FROM python:3.11-slim

ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 5000

CMD ["gunicorn", "--bind", "0.0.0.0:5000", "app:app"]
```

```
mon-projet
├── frontend
│   └── Dockerfile
├── backend
│   ├── app.py
│   ├── requirements.txt
│   └── Dockerfile
├── nginx
│   └── default.conf
└── docker-compose.yml
```

Multi-stage build

```
FROM node:18 AS builder

WORKDIR /app

COPY package.json package-lock.json ./
RUN npm install

COPY . .
RUN npm run build

FROM nginx:alpine

RUN rm -rf /usr/share/nginx/html/*

COPY --from=builder /app/build /usr/share/nginx/html

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

Architecture



SYNTHESE : Analyse comparative

Élément	DEV	PROD
React	npm start	build + nginx
Flask	python app.py	gunicorn
Ports	multiples	80
Optimisation	non	oui
Reverse proxy	non	nginx

Commandes essentielles

docker compose up

→ Démarrer tous les services

docker compose up -d

→ Démarrer tous les services mais en arrière plan (on récupère le terminal)

docker compose down

→ Arrête et supprime les conteneurs et les réseaux créés (mais pas les volumes)

docker compose down -v

→ Arrête et supprime les conteneurs , les réseaux créés et les volumes

docker compose build

→ Crée toutes les images nécessaires

docker compose build --no-cache

→ Permet de reconstruire les images sans cache

docker compose logs

→ ¨Permet de voir les logs

docker compose ps

→ Pour voir tous les services actifs

docker compose rm

→ Supprimer tous les conteneurs

docker compose start

→ Démarrer les conteneurs

docker compose stop

→ Arrêter les conteneurs

docker compose restart

→ Redémarrer les services

Commandes essentielles

```
docker compose exec « nom du service » bash
```

→ Exécuter une commande dans un conteneur

```
docker compose pull « nom de l'image »
```

→ Télécharger une image depuis Docker Hub

```
docker compose up --build
```

→ Construit les images et démarre les conteneurs

Atelier Pratique

Déploiement d'une Application Web avec Docker Compose

Merci

Y. NDIONE

youcomputing@gmail.com